Team TenderMatch

Team Members: Vikram Jirgale, Adwyte Karandikar, Raj Kakade, Atharva Kangralkar, and Prajwal Kadam

College: Vishwakarma Institute of Technology, Pune

Project Github Link: https://github.com/Prajwalkadam29/TenderMatch-Phase-II

1. Executive Summary

TenderMatch is designed to reduce the manual effort involved in finding and evaluating tenders by converting a fragmented procurement process into a structured, machine-readable workflow. The project addresses a common operational pain point: vendors must read long tender documents, interpret eligibility clauses, compare requirements with their own business profile, and then decide whether to respond. In practice, this is slow, repetitive, and error-prone.

The current phase of the project moves beyond simple keyword search. It introduces a structured vendor profile model, a normalized tender store, and a rule-driven matching pipeline that can compare the two with explainable scoring. The design also supports multi-tenant access, role-based access control, and a clean data separation strategy using MongoDB collections for users, organizations, vendor profiles, tenders, and match results.

From the work completed so far, the project has already reached an important architectural milestone. The data model is defined, the primary business entities are identified, the external tender source strategy has been analyzed, and the Bidassist Public API has been selected as a practical structured source for tender and tender-result retrieval. This gives the project a reliable foundation for the next implementation phase.

2. Problem Analysis and Project Direction

The problem statement shows that tender discovery is not just a search problem. It is a document understanding and decision-support problem. Government and enterprise tenders are often published in large volumes across multiple portals, and the documents themselves can be extremely long, technical, and inconsistent in structure. A vendor can easily miss relevant opportunities simply because the tender is too large to read in time or because the relevant criteria are buried in dense eligibility sections.

The project therefore needs to solve three separate challenges at the same time. First, it must collect tender data from dependable sources. Second, it must represent vendor capabilities in a structured way so they can be matched programmatically. Third, it must explain why a tender is or is not suitable, so that users can trust the result rather than treat it as a black box.

Our solution combines structured databases, deterministic business rules, and AI-assisted interpretation. This hybrid approach is important because procurement decisions often depend on hard constraints such as location, certification, turnover, and eligibility requirements. These constraints must be enforced strictly before softer semantic matching is applied.

3. What Has Been Done So Far

3.1 Requirement study and portal analysis

A major part of the early work was analyzing tender websites to understand where tenders can be sourced and what obstacles exist in each source. The research compared government portals and private portals, and noted whether they require captchas, subscriptions, registration details, or have an available API. This analysis is important because it directly affects how the ingestion pipeline will be built.

The portal study showed that some sources are useful for discovery but are difficult to automate because of login barriers or captchas. In contrast, Bidassist stood out because it provides an official public API with endpoints for searching tenders and tender results. That makes it a much better fit for a structured backend than fragile page scraping alone.

Tender site research sheet:

https://docs.google.com/spreadsheets/d/1tg6u6mmG1vM5yao-iycYYTIYfAWrXr7jkA05itvX29Y/edit?gid=0#gid=0

3.2 Source selection and ingestion strategy

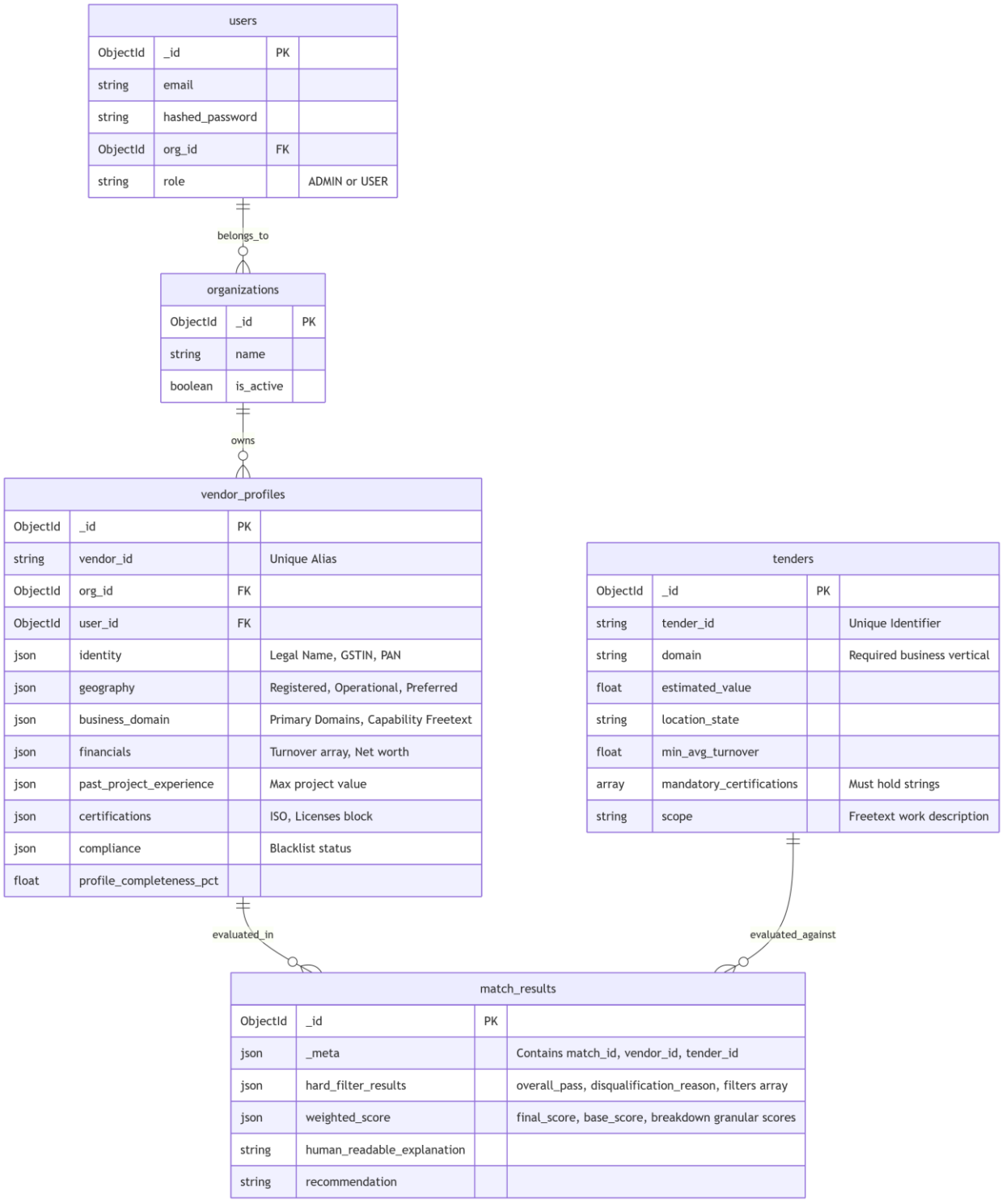
The project currently follows a **two-track sourcing** idea. One track is direct **API-based retrieval** for structured tender data. The second track is **web extraction support** for portals that do not expose clean APIs. This is why the Firecrawl service was documented as a retrieval bridge: it can fetch webpage content, normalize it, and hand it to parsing logic.

At the same time, the Bidassist Public API is now the most important structured source because it can provide programmatic access to tender records and tender results. This reduces dependence on brittle scraping and helps the project move toward a more stable, scalable backend.

3.3 Database schema design

The database schema is one of the strongest parts of the current design. The entities are separated by responsibility instead of mixing everything into one large collection. The `**users**` collection stores authentication and tenancy data, `**organizations**` stores company-level ownership, vendor_profiles stores the detailed vendor capability model, `**tenders**` stores normalized tender requirements, and `**match_results**` stores the outcomes of each evaluation.

The schema also shows clear relationships between the entities. A user belongs to one organization. An organization owns vendor profiles. A vendor profile is evaluated against one or more tenders. Each evaluation is recorded as a match_result so the system can keep a historical audit trail. This is the right structure for traceability, debugging, and future learning features.



3.4 Vendor profile model

The vendor profile design is intentionally detailed because the matching engine can only be as good as the data it receives. The profile captures identity and compliance information, business domains, geography, financial strength, project experience, certifications, and compliance status. It also stores a profile completeness percentage, which can later be used to reward richer profiles in the scoring system.

This structure is important because it turns a vendor from a vague text description into a queryable business object. Instead of searching a brochure, the system can ask direct questions such as: Does this vendor operate in the required state? Do they hold the required certification? Is their turnover above the minimum threshold? Have they completed projects of comparable value?

3.5 Tender model

The tender collection captures the fields needed for matching: tender identifier, domain, estimated value, required business vertical, location state, minimum average turnover, mandatory certifications, and the work scope. This makes the tender ready for deterministic comparison against vendor profiles.

The design choice here is practical. By storing tenders in a normalized structure, the system can evaluate many vendors against many tenders without repeatedly parsing raw documents. This also makes the future bulk-evaluation workflow much easier to implement.

3.6 Match result model

The match_results collection is designed for explainability. It stores metadata such as the match id, vendor id, and tender id, along with hard filter outcomes, weighted score breakdowns, the final recommendation, and a human-readable explanation. This is essential for procurement use cases because users need to know why the system made a recommendation.

The presence of a structured result document also means the application can later build dashboards, audit trails, and feedback loops without recomputing everything from scratch.

3.7 Current core collections

Collection	Purpose	Key idea
users	Stores login identity and access role	Links each account to one organization
organizations	Represents the tenant/company	Acts as the owner of vendor profiles
vendor_profiles	Stores structured vendor capability data	Used as the input to matching
tenders	Stores normalized tender requirements	Used as the input to evaluation
match_results	Stores scoring and explanation output	Creates an audit trail of decisions

4. Matching Logic and Evaluation Flow

The project's matching layer is built as a staged decision system. This is a strong design choice because procurement matching should not depend only on similarity scores. Some tender requirements are absolute, meaning the vendor either qualifies or does not qualify. Other requirements are better treated as ranked preferences. The system therefore separates hard rejection rules from weighted scoring.

4.1 Hard eligibility filters

The first stage is a binary gate. If the vendor fails any critical requirement, the match should not proceed. Typical hard filters include blacklist or debarment checks, domain mismatch, missing mandatory certifications, location mismatch, and insufficient turnover. This protects the system from producing misleading scores for vendors who are not legally or practically eligible.

This stage is especially important for user trust. A vendor may have a very good textual similarity score, but still be ineligible if the tender requires a specific certification or minimum financial capacity. The hard filter stage makes the system safer and more defensible.

4.2 Weighted scoring stage

If the vendor passes the hard filters, the system can then compute a weighted score across dimensions such as domain fit, geography, financial capacity, project experience, certifications, semantic capability match, and compliance risk. This gives the project a balanced view of fit instead of a single yes/no answer.

The semantic component is particularly useful when the vendor profile contains a free-text capability description. Instead of relying only on exact keyword overlap, the system can compare the meaning of the vendor's description with the tender scope. That is useful when wording differs but intent is similar.

4.3 Explainable output

The final output is not just a number. It should also include a human-readable explanation and a breakdown of the strongest and weakest factors. This is important because procurement decisions are reviewed by people, and they need an answer that can be understood quickly.

The match_results schema already reflects this idea by storing weighted_score, hard_filter_results, human_readable_explanation, and recommendation. That means the project is being built with explainability as a first-class feature rather than as an afterthought.

5. Current Architecture and Technical Stack

The backend is being developed in Python using FastAPI, which is a suitable choice for asynchronous APIs and modular service design. The architecture follows a service-oriented structure with separate layers for clients, schemas, and services. This separation makes the codebase easier to maintain and test.

MongoDB is being used as the database because the project deals with flexible, nested documents rather than rigid relational rows. That fits vendor profiles especially well, since certificates, geographies, financial entries, and project histories may all be variable-length arrays or nested objects.

The frontend is built with React and Vite, with TypeScript and Tailwind CSS for type safety and fast UI development. The documentation and UI direction suggest a modern dashboard-style experience, which is appropriate for onboarding vendors, viewing matches, and showing detailed result cards.

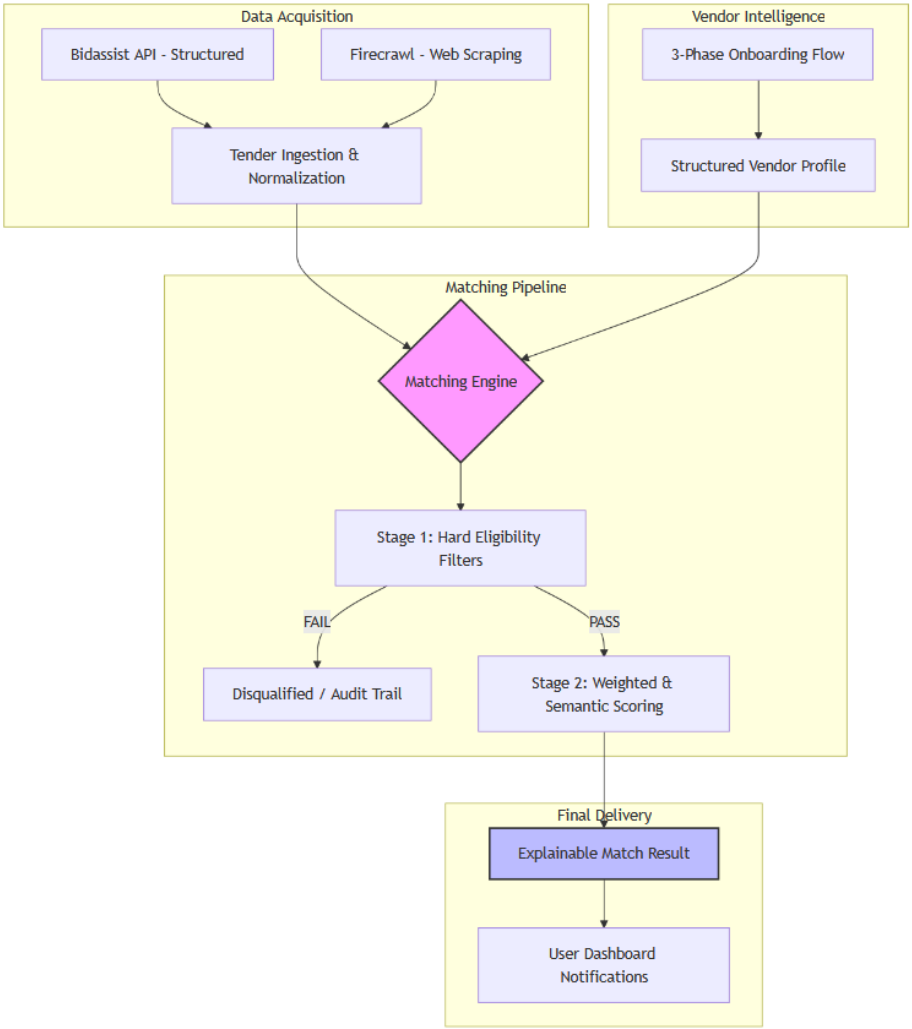


Figure: Project Flow Diagram

6. How the Project Moves Forward

6.1 Complete structured tender ingestion

The next step is to make tender ingestion dependable and repeatable. The project should first finalize the Bidassist integration because it provides a cleaner and more scalable data source than many scraping-only portals. Once the API ingestion path is stable, the system can normalize tender data into the `**tenders**` collection automatically.

After that, **Firecrawl** or similar extraction services can remain as secondary support for portals that need webpage fetching or where the API does not provide enough coverage. This creates a flexible ingestion layer instead of a single-source dependency.

6.2 Strengthen vendor onboarding

The vendor profile builder should be completed as a guided multi-step flow so that users can submit clean and rich data. The better the profile quality, the more accurate the matching result will be. This is why the profile completeness metric is useful: it encourages better inputs and can improve ranking quality.

Validation should remain strict. For example, structured fields like turnover, certification names, and states should be stored in normalized formats, while free-text fields can be used for semantic matching.

6.3 Finalize the matching dashboard

The matching interface should allow a vendor profile to be selected and then evaluated against a batch of tenders. The user should see clear cards or rows for each tender, along with status indicators such as pass, fail, or partially suitable. Every result should include the reason for the score so that users can review it immediately.

This dashboard layer is important because it turns the backend algorithm into a usable business product. Without a clear interface, even a strong matching engine would be difficult to adopt.

6.4 Add learning and feedback loops

Once the first version is stable, the project can start capturing feedback such as interested, not relevant, submitted, won, or lost. That feedback can later be used to improve the scoring weights, tune thresholds, or train future recommendation logic.

This is also the point where the project can evolve toward more agentic behavior. For example, one module can handle tender parsing, another can handle vendor matching, and another can handle notifications or follow-up messaging.

6.5 Future enhancements

Future phases can include WhatsApp and email alerts, multilingual messages, stronger analytics dashboards, and a richer AI reasoning layer. A real-time risk assessment engine can also be added later to evaluate vendor stability using external financial or news signals before a match is confirmed.

These enhancements would move the system from a matching tool into a full procurement intelligence platform. The current design already supports that direction because the data model and service structure are modular.

- **WhatsApp and email alert system:** Build personalized tender notifications using the WhatsApp Business API and email delivery. Messages will include the tender title, fit score, top matching reasons, deadline, and a direct link to the match result. Vendors will be able to respond with a single reply to mark interest or dismiss the alert.
- **Multilingual message support:** Extend notification templates to support English, Hindi, Marathi, and Tamil. A lightweight LangChain translation chain will convert the generated English message into the vendor's preferred language at delivery time, without requiring separate templates for each language.
- **Analytics dashboard for performance monitoring:** Add a dedicated admin dashboard panel showing key metrics such as match accuracy rate, notification delivery rate, average fit score distribution, vendor engagement rate, and the number of tenders won or submitted after a recommendation. Metrics will update in near real time using the match_results collection.
- **Real-time vendor risk assessment engine:** Integrate external financial data signals and news feeds to evaluate vendor stability before confirming a high-value match. The engine will flag vendors with recent adverse news, credit downgrades, or lapsed certifications, and will surface a risk indicator alongside the fit score in the dashboard.
- **Deeper AI reasoning layer using LangGraph agentic workflows:** Evolve the matching pipeline into a fully autonomous multi-agent system where separate agents handle tender ingestion, document parsing, eligibility filtering, weighted scoring, explanation generation, and notification dispatch. Each agent operates independently within a shared state graph, enabling parallel processing, conditional branching, and self-correcting retries without manual intervention.
- **Vendor feedback learning loop:** Capture structured feedback from vendors (Interested, Not Relevant, Submitted, Won, Lost) and use it to automatically adjust scoring dimension weights over time. Vendors who consistently win after a domain-fit match will cause that dimension's weight to increase, making future recommendations progressively more accurate without requiring manual tuning.
- **Tender document auto-summarisation:** Add a one-click summarisation feature in the dashboard that generates a concise two-page brief of any tender document, highlighting the scope of work, key eligibility criteria, financial requirements, and critical deadlines. This saves vendors from reading lengthy original documents before deciding whether to pursue an opportunity.
- **Multi-vendor batch evaluation:** Allow an organisation to evaluate multiple vendor profiles against a single tender simultaneously, producing a ranked shortlist. This is especially useful for consultants or procurement aggregators who manage several vendor clients and need to identify the best-fit candidate quickly.
- **Deadline tracking and calendar integration:** Implement automated reminders for upcoming tender submission deadlines based on the dates stored in the tenders collection. Vendors will

receive a reminder notification 7 days and 2 days before the deadline for any tender they have marked as Interested, with optional calendar event export.

- **Admin dashboard for monitoring and control:** Build a dedicated admin panel covering vendor onboarding status, notification delivery logs, match pipeline execution history, and scoring configuration. Admins will be able to manually trigger re-evaluation of a vendor-tender pair, adjust global scoring weights, and review flagged matches before they are dispatched.

6.6 Agentic AI Implementation Using LangChain and LangGraph

The next major evolution of TenderMatch is converting its pipeline into a fully agentic system using LangChain and LangGraph. Instead of a fixed sequential flow, each major function becomes an autonomous agent node in a stateful graph. The agents share a common state object, make decisions independently, and hand off control to the next appropriate node. This architecture enables parallel execution, conditional branching, human-in-the-loop checkpoints, and self-correcting retry logic. The following steps outline the exact implementation plan.

Step 1: Define the shared agent state

Create a typed Python dataclass or TypedDict called TenderMatchState using LangGraph's StateGraph. This state object acts as the shared memory passed between every agent node in the graph. It must include fields for: the raw tender source URL or file path, the extracted tender JSON, the vendor profile ID being evaluated, the hard filter results, the weighted score breakdown, the final recommendation, the human-readable explanation, and the notification status. Every agent reads from this state and writes its output back into it. Using LangGraph's built-in state persistence, the graph can be paused and resumed at any checkpoint, which is critical for human-in-the-loop review steps.

Step 2: Build the Tender Ingestion Agent

Implement a LangGraph node called tender_ingestion_agent. This node is responsible for deciding how to retrieve a tender based on the source type. It wraps two LangChain tools: a BidassistAPITool that calls the Bidassist public API endpoints for structured tender data, and a FirecrawlTool that fetches webpage content from portals without clean APIs. The node uses an LLM-backed router (implemented as a LangChain RunnableWithFallbacks) to choose the correct tool based on the source URL pattern. After retrieval, the raw content is stored in the state for the next agent to process.

Step 3: Build the Tender Parsing Agent

Implement a LangGraph node called tender_parsing_agent. This node receives the raw tender content from the state and uses a LangChain LLM chain with a structured output parser to extract: tender ID and title, estimated value, required domain and business vertical, geographic location and state, minimum average annual turnover, mandatory certifications, scope of work, submission deadline, and eligibility clauses. The output parser uses Pydantic models to enforce the schema and returns a validated TenderSchema object. This normalized output is written back to the state as extracted_tender. The node retries up to three times with a fallback prompt if parsing fails on the first attempt.

Step 4: Build the Hard Filter Agent

Implement a LangGraph node called hard_filter_agent. This node is purely deterministic and does not use an LLM. It reads the extracted tender and the vendor profile from the state, then evaluates five binary rules in sequence: (1) blacklist and debarment check against the vendor's compliance status, (2) domain match between the tender's required vertical and the vendor's registered business sectors, (3) geographic match between the tender's required state and the vendor's operational geography, (4) turnover check comparing the tender's minimum annual turnover requirement against the vendor's audited figures, and (5) mandatory certification check verifying that all required certifications are present in the vendor's profile. If any rule fails, the node sets hard_filter_passed to false and writes the reason to the state. LangGraph's conditional edge is then used to route failed vendors directly to the notification agent with a rejection message, skipping the scoring stage entirely.

Step 5: Build the Scoring Agent

Implement a LangGraph node called scoring_agent. This node runs only when the hard filter has passed. It computes a weighted fit score from 0 to 100 across seven dimensions: domain fit (25%), geographic match (15%), financial capacity (20%), project experience and track record (15%), certifications and compliance (10%), semantic capability match (10%), and compliance risk (5%). The semantic capability match dimension uses a LangChain embedding comparison between the vendor's free-text capability description and the tender's scope of work, using cosine similarity on OpenAI or sentence-transformer embeddings stored in a vector store. The final weighted score and per-dimension breakdown are written to the state as score_breakdown.

Step 6: Build the Explanation Agent

Implement a LangGraph node called explanation_agent. This node takes the score breakdown and hard filter results from the state and uses a LangChain LLMChain with a structured prompt to generate a concise, human-readable explanation of the match result. The prompt instructs the model to write two to three sentences summarizing the strongest matching factors, one sentence on the weakest area, and a final recommendation (Strongly Recommended, Recommended, Partially Suitable, or Not Eligible). This explanation is stored in the state as human_readable_explanation and is also saved to the match_results MongoDB collection along with the full score breakdown and metadata, creating a permanent audit trail.

Step 7: Build the Notification Agent

Implement a LangGraph node called notification_agent. This node is triggered by two paths: successful matches with a score above a configurable threshold (default 60), and hard filter rejections that the vendor has opted in to receive. The node uses LangChain tools to dispatch alerts: a WhatsAppTool wrapping the WhatsApp Business API and an EmailTool wrapping an SMTP or SendGrid client. Message templates are generated dynamically using a LangChain PromptTemplate that inserts the tender title, fit score, top three matching factors, and a call-to-action link. Multilingual support is implemented by passing the vendor's preferred language to the template, with translations handled by a lightweight LangChain translation chain using a smaller and faster model. Delivery status is written back to the state and stored in the match_results collection.

Step 8: Wire the LangGraph graph and define edges

Use LangGraph's StateGraph to assemble all agent nodes and define the execution graph. Add nodes in this order: tender_ingestion_agent, tender_parsing_agent, hard_filter_agent, scoring_agent, explanation_agent, and notification_agent. Add a standard directed edge from ingestion to parsing, and from parsing to hard_filter. Add a LangGraph conditional edge from hard_filter: if hard_filter_passed is false, route directly to notification_agent; if true, route to scoring_agent. Add standard edges from scoring to explanation, and from explanation to notification. Set tender_ingestion_agent as the entry point using graph.set_entry_point(). Compile the graph with a LangGraph MemorySaver checkpointer so that state is persisted after each node completes. This allows the pipeline to be interrupted for human review after hard filtering and before sending notifications, if that checkpoint is configured.

Step 9: Integrate a feedback loop agent

Add a separate LangGraph subgraph called feedback_loop_graph that runs asynchronously from the main matching pipeline. This subgraph listens for vendor feedback events (Interested, Not Relevant, Submitted, Won, Lost) captured via the dashboard or WhatsApp reply. A feedback_collection_node receives the event and updates the corresponding match_results document in MongoDB. A weight_adjustment_node then computes updated scoring weights by analysing patterns across recent feedback: for example, if vendors with a high financial capacity score are consistently winning, that dimension's weight is nudged upward. The adjusted weights are stored in a scoring_config collection and read by the scoring_agent on its next run. This closes the learning loop without requiring a full model retraining cycle.

Step 10: Expose the graph through the FastAPI backend

Wrap the compiled LangGraph graph in a FastAPI service with three endpoints. The first endpoint, POST /match/run, accepts a vendor profile ID and a list of tender IDs, invokes the graph for each pairing asynchronously using Python's asyncio, and returns a list of match result IDs. The second endpoint, GET /match/{result_id}, retrieves a completed match result including score breakdown and explanation from MongoDB. The third endpoint, POST /match/feedback, accepts a vendor feedback event and publishes it to the feedback_loop_graph. The existing React frontend connects to these endpoints to power the matching dashboard, result cards, and feedback buttons. Graph execution traces are logged using LangSmith for debugging and performance monitoring.

7. Documentation-level assumptions and design decisions

One practical assumption behind the current design is that tender data should be normalized before scoring. This is important because raw documents are noisy and often inconsistent. By converting them into structured fields first, the matching logic becomes simpler, faster, and easier to explain.

Another key decision is to keep AI support as an assistant rather than the sole decision-maker. AI is valuable for interpreting free text, summarizing scope, and creating explanation text, but eligibility decisions should still be controlled by deterministic rules where possible. That balance makes the system more reliable.

The final assumption is that explainability is not optional. In procurement, a recommendation must be defensible. That is why the schema already stores the score breakdown and human-readable explanation. It gives the team a clean path toward an auditable product.

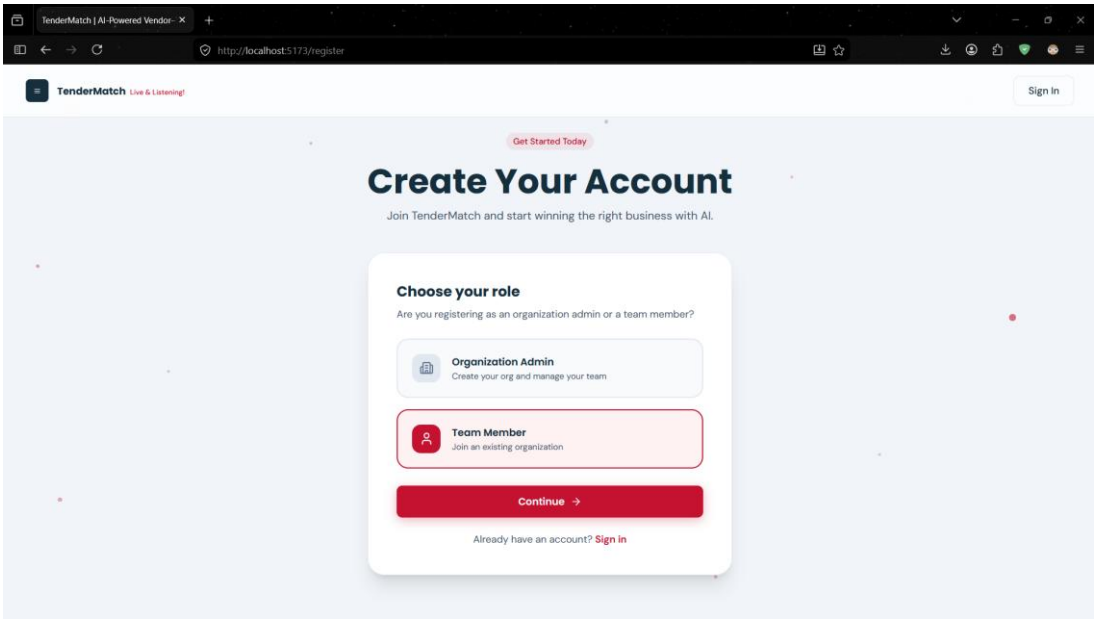
8. System Interface & Visualizations

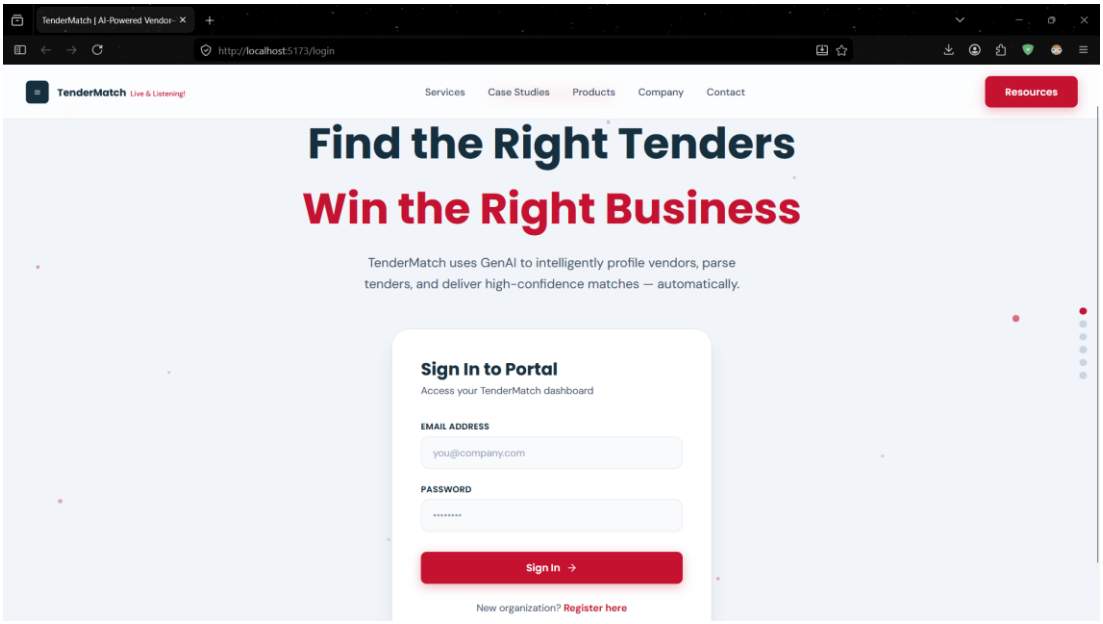
This section provides a visual overview of the **TenderMatch** platform, including the user interface and the underlying data architecture.

The frontend utilizes **React** and **Tailwind CSS** to deliver a modern, responsive dashboard experience. The entry point for all users is the secure login portal, which serves as the gateway to the multi-tenant environment.

9.1 User Authentication Portal

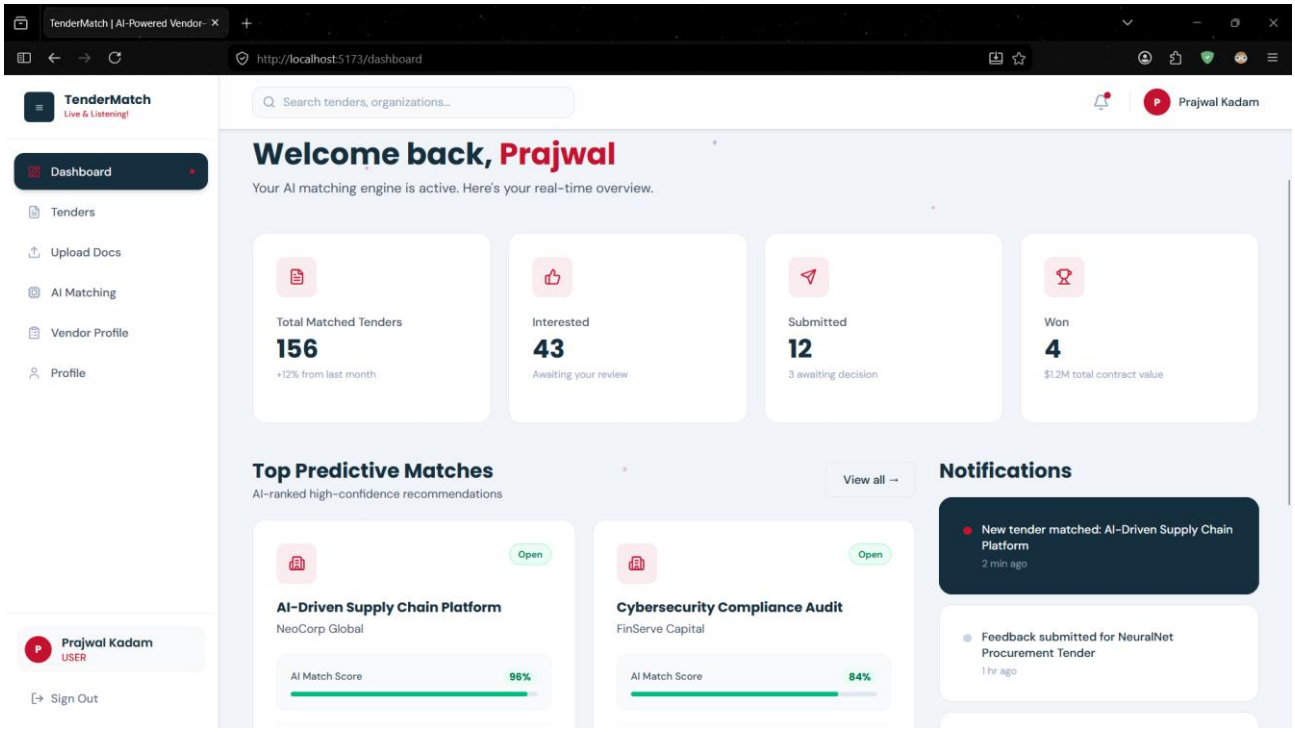
The entry point for the multi-tenant environment





9.2 Centralized Analytics Dashboard

The main dashboard provides a real-time overview of the vendor's procurement pipeline.



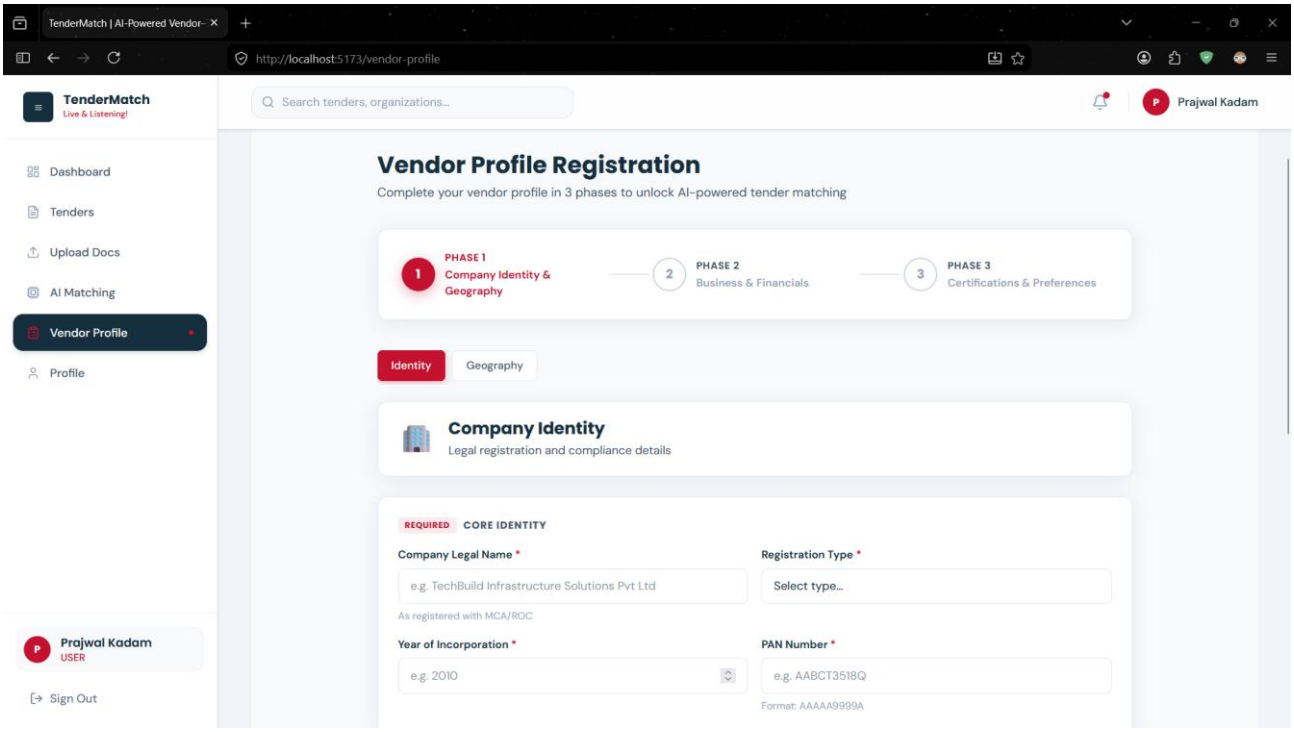
The TenderMatch User Dashboard

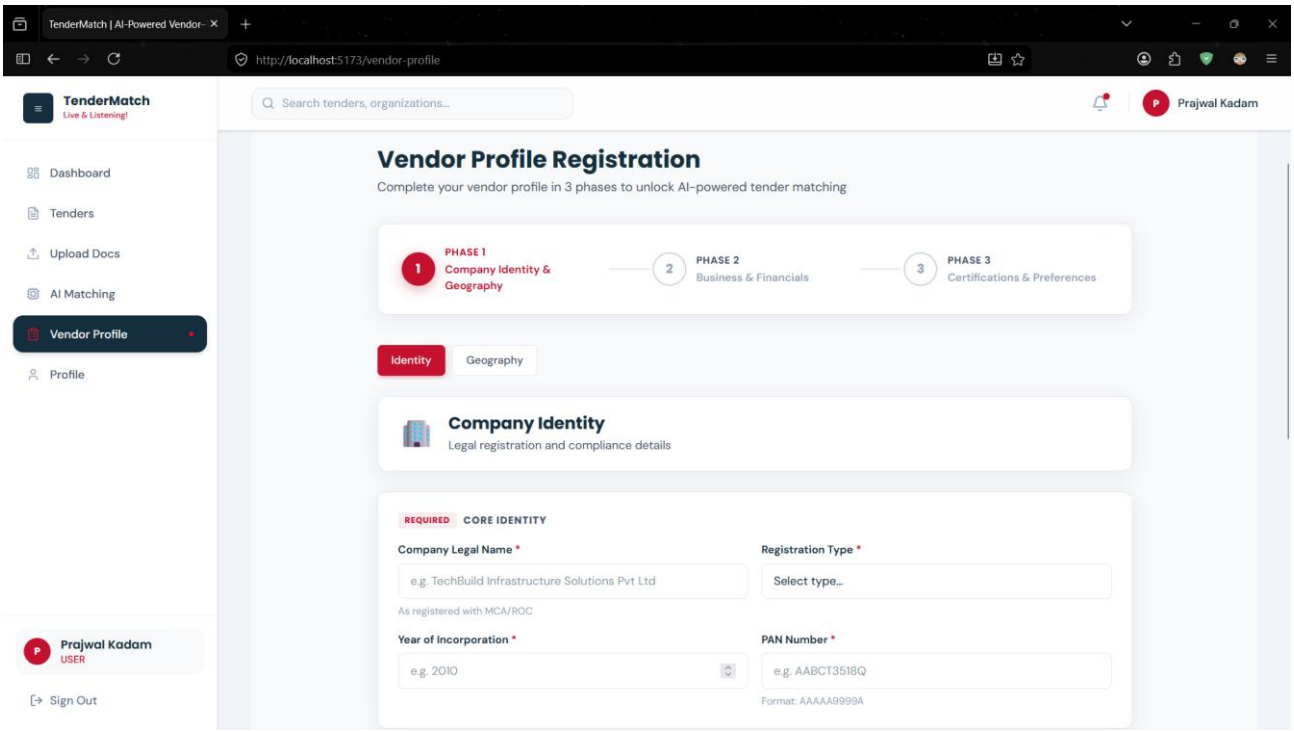
9.3 Guided Vendor Onboarding & Profile Registration

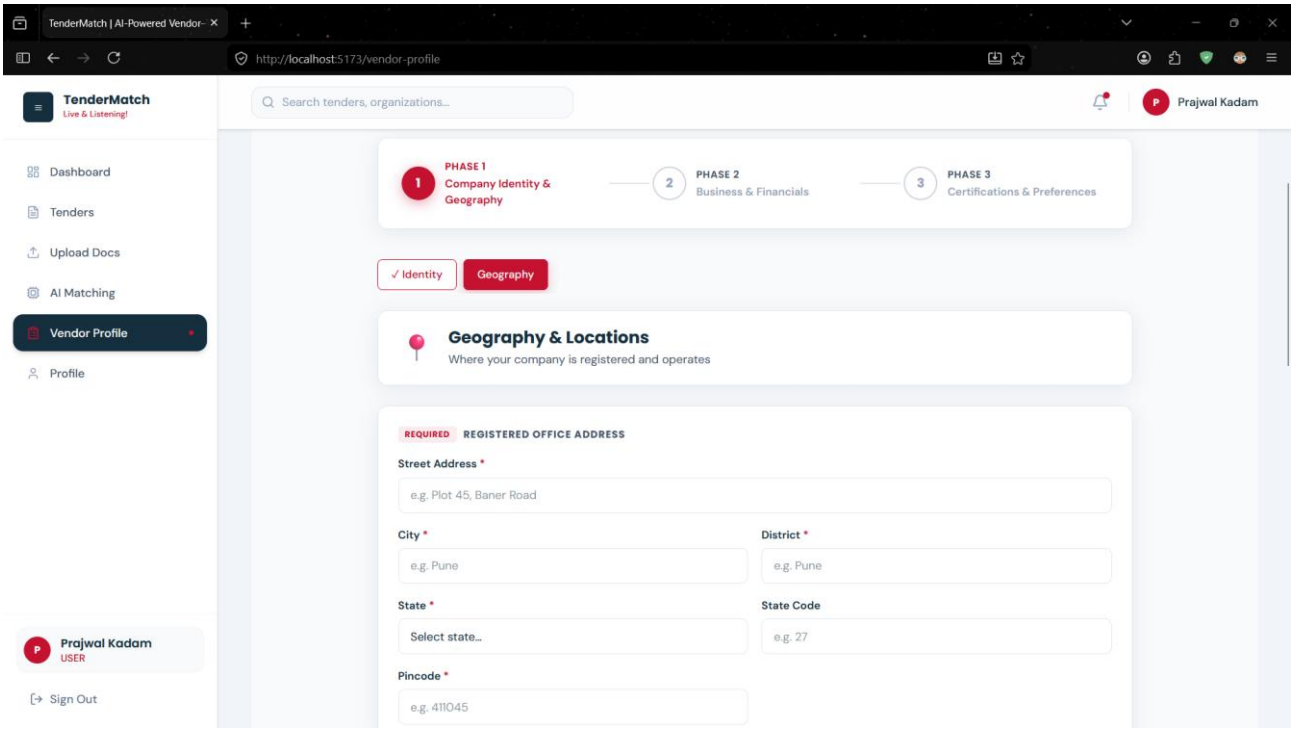
To ensure high data quality and improved matching accuracy, the system implements a structured, three-phase registration process. This flow enforces strict validation on financial and compliance fields while allowing flexibility for domain-specific data.

9.3.1 Phase 1: Company Identity & Geography

This phase establishes the vendor's legal foundation and operational footprint. It captures essential identifiers such as **PAN** and **Registration Type**, alongside the **Registered Office Address**, which is critical for deterministic location-based filtering.

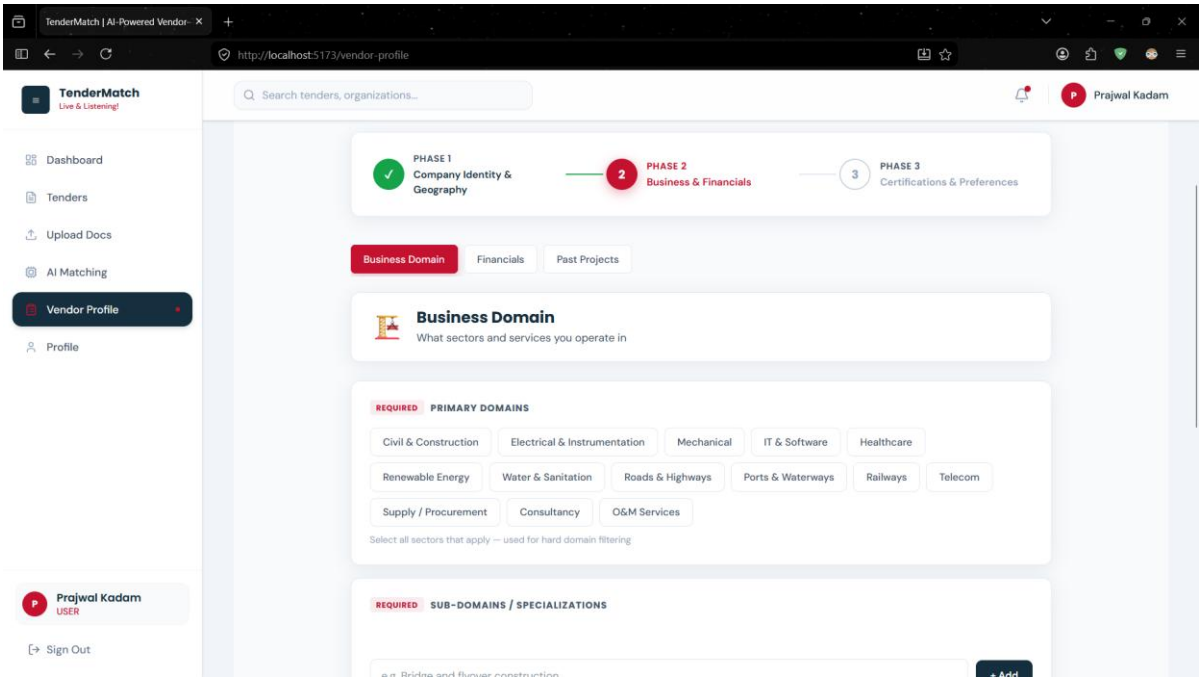

TenderMatch | AI-Powered Vendor-
http://localhost:5173/vendor-profile
TenderMatch
Live & Listening!
Search tenders, organizations...
Prajwal Kadam
Dashboard
Tenders
Upload Docs
AI Matching
Vendor Profile
Profile
Vendor Profile Registration
Complete your vendor profile in 3 phases to unlock AI-powered tender matching
1
PHASE 1
Company Identity & Geography
2
PHASE 2
Business & Financials
3
PHASE 3
Certifications & Preferences
Identity
Geography
Company Identity
Legal registration and compliance details
REQUIRED
CORE IDENTITY
Company Legal Name *
e.g. TechBuild Infrastructure Solutions Pvt Ltd
As registered with MCA/ROC
Registration Type *
Select type...
Year of Incorporation *
e.g. 2010
PAN Number *
e.g. AABCT3518Q
Format: AAAAA9999A
Prajwal Kadam
USER
Sign Out

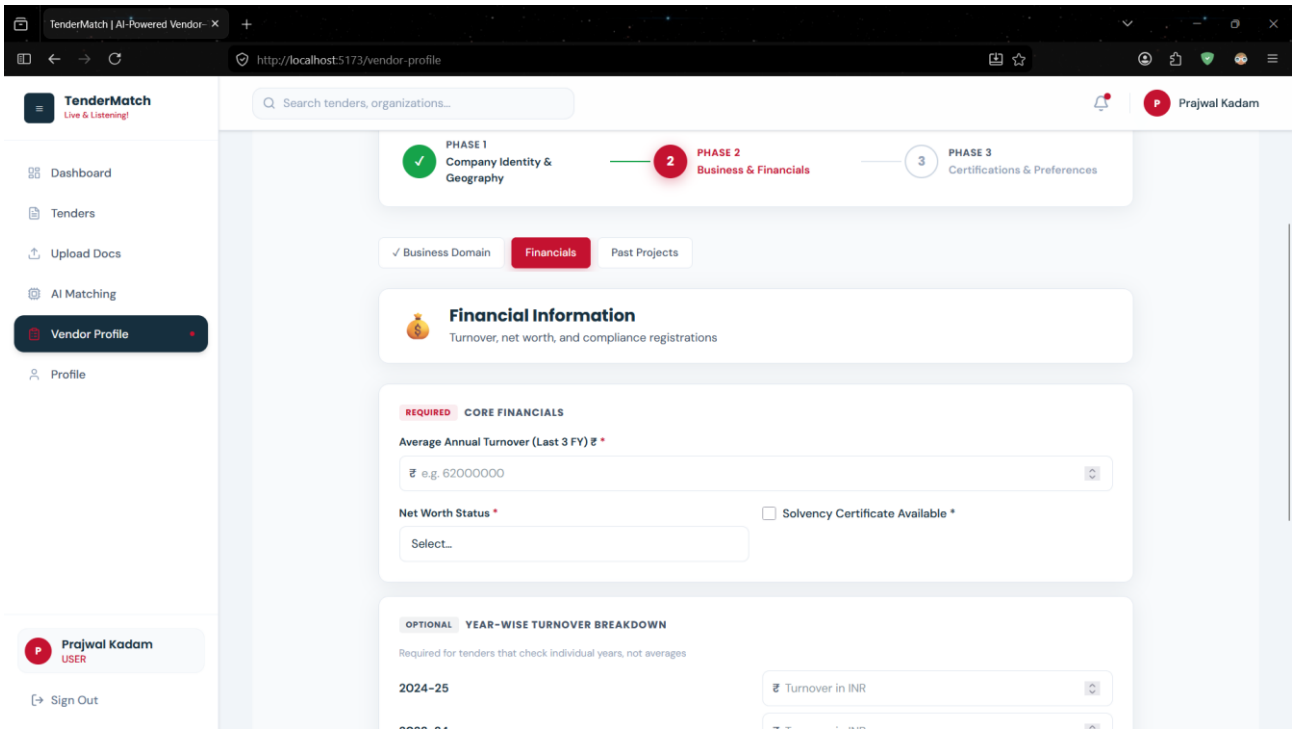

TenderMatch | AI-Powered Vendor-
http://localhost:5173/vendor-profile
TenderMatch
Live & Listening!
Search tenders, organizations...
Prajwal Kadam
Dashboard
Tenders
Upload Docs
AI Matching
Vendor Profile
Profile
Vendor Profile Registration
Complete your vendor profile in 3 phases to unlock AI-powered tender matching
1
PHASE 1
Company Identity & Geography
2
PHASE 2
Business & Financials
3
PHASE 3
Certifications & Preferences
Identity
Geography
Company Identity
Legal registration and compliance details
REQUIRED
CORE IDENTITY
Company Legal Name *
e.g. TechBuild Infrastructure Solutions Pvt Ltd
As registered with MCA/ROC
Registration Type *
Select type...
Year of Incorporation *
e.g. 2010
PAN Number *
e.g. AABCT3518Q
Format: AAAAA9999A
Prajwal Kadam
USER
Sign Out

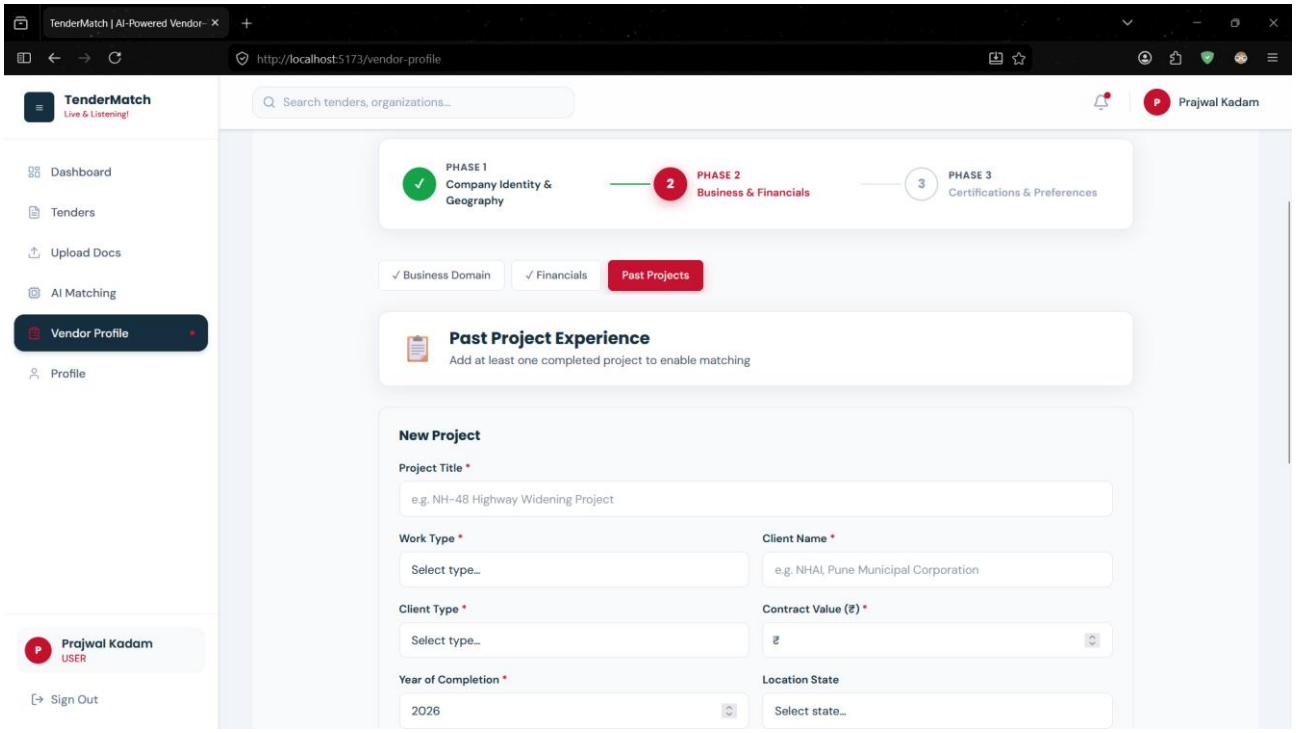


9.3.2 Phase 2: Business Domains & Financial Strength

This phase enables the engine to perform hard domain filtering and financial capacity checks. Vendors select from primary sectors (e.g., Civil & Construction, IT & Software) and provide audited turnover data to meet minimum tender thresholds.

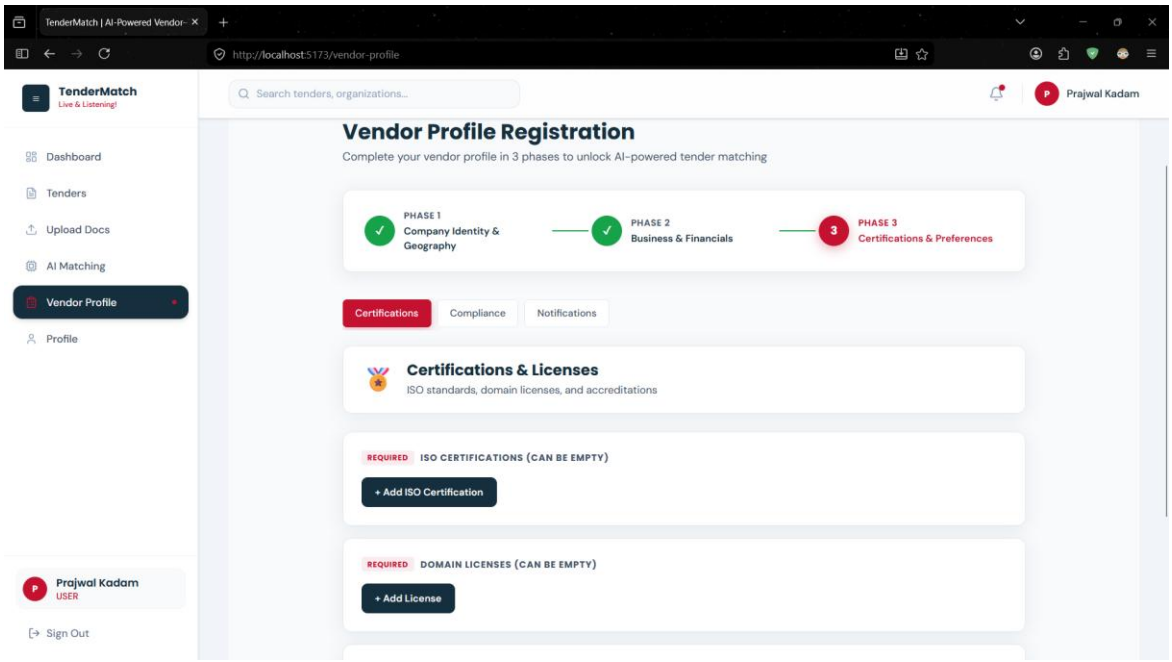



TenderMatch | AI-Powered Vendor-
http://localhost:5173/vendor-profile
TenderMatch
Live & Listening!
Search tenders, organizations...
Prajwal Kadam
Dashboard
Tenders
Upload Docs
AI Matching
Vendor Profile
Profile
PHASE 1
Company Identity & Geography
PHASE 2
Business & Financials
PHASE 3
Certifications & Preferences
✓ Business Domain
Financials
Past Projects
Financial Information
Turnover, net worth, and compliance registrations
REQUIRED
CORE FINANCIALS
Average Annual Turnover (Last 3 FY) ₹ *
₹ e.g. 62000000
Net Worth Status *
Select...
Solvency Certificate Available *
OPTIONAL
YEAR-WISE TURNOVER BREAKDOWN
Required for tenders that check individual years, not averages
2024-25
₹ Turnover in INR
Prajwal Kadam
USER
Sign Out

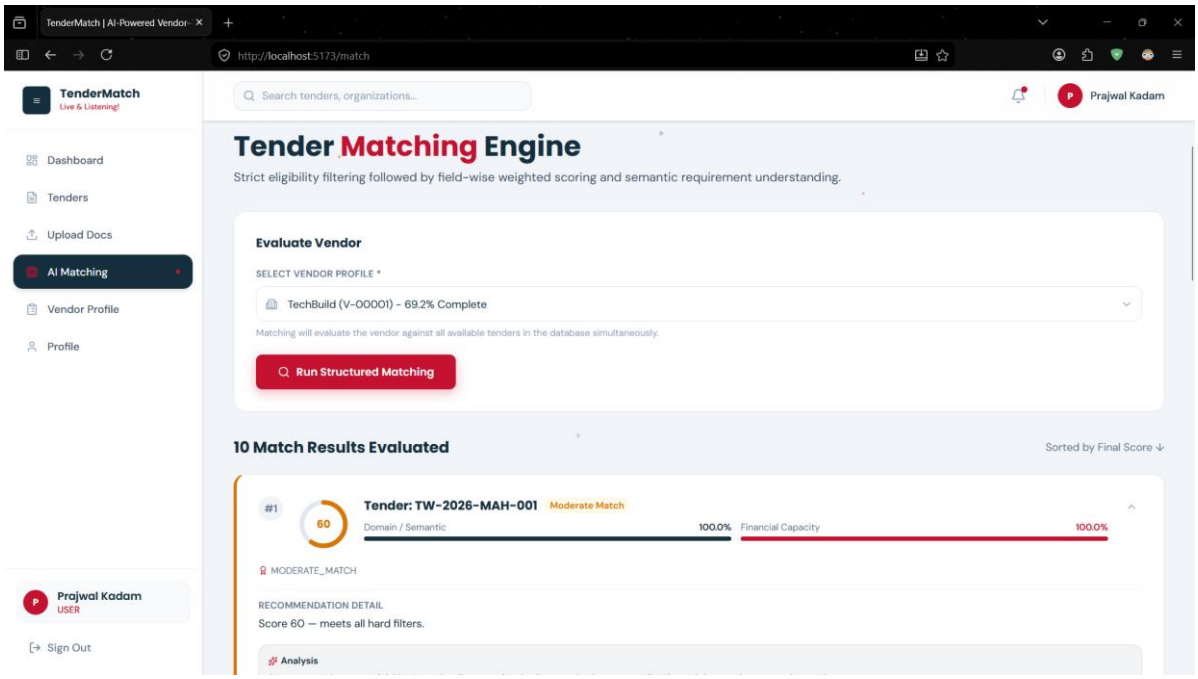

TenderMatch | AI-Powered Vendor-
http://localhost:5173/vendor-profile
TenderMatch
Live & Listening!
Search tenders, organizations...
Prajwal Kadam
Dashboard
Tenders
Upload Docs
AI Matching
Vendor Profile
Profile
PHASE 1
Company Identity & Geography
PHASE 2
Business & Financials
PHASE 3
Certifications & Preferences
✓ Business Domain
✓ Financials
Past Projects
Past Project Experience
Add at least one completed project to enable matching
New Project
Project Title *
e.g. NH-48 Highway Widening Project
Work Type *
Select type...
Client Name *
e.g. NHAI, Pune Municipal Corporation
Client Type *
Select type...
Contract Value (₹) *
₹
Year of Completion *
2026
Location State
Select state...
Prajwal Kadam
USER
Sign Out

9.3.3 Phase 3: Certifications & Past Performance

The final phase captures mandatory certifications (e.g., ISO standards) and past project experience. These entries are stored as flexible documents in **MongoDB**, allowing the system to handle variable-length project histories and diverse license types.



9.4 AI Matching & Result Explainability



The AI Matching Dashboard, where a selected vendor profile is evaluated against the tender store.

8. Conclusion

TenderMatch has a clear and well-structured foundation. The problem has been identified correctly, the data model has been designed carefully, the tender source landscape has been studied, and a practical API-first sourcing strategy has been selected. The current architecture also supports explainable matching, which is essential for procurement workflows.

In simple terms, the project is moving in this direction: first organize the data, then normalize tender sources, then compare vendor capability with tender requirements in a transparent way, and finally build notification and learning layers on top. This makes the project technically sound and also useful in a real business environment.